Project

CSE 4224: Digital System Design Laboratory

Electronic Voting Machine (EVM)

By

Sk. Azraf Sami

Roll: 1907115

&

Saugata Roy Arghya

Roll: 1907116

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

December, 2024

1 Introduction

The Electronic Voting Machine (EVM) is a digital voting system implemented using Verilog, designed to securely and efficiently simulate an election process. The machine supports voting for multiple candidates, ensures vote authenticity, provides real-time vote counts, and declares results upon the end of voting. Additionally, it incorporates an admin-controlled reset for new sessions.

2 Project Description

2.1 Features

- **Multiple Candidates Voting**: Enables secure voting for multiple candidates through a user-friendly interface.
- **Vote Authentication:**: Implements voter authentication to ensure only valid votes are cast.
- **Real-Time Vote Counting**: Displays vote counts dynamically as votes are recorded.
- **Admin Control**: Includes features for starting, ending, and resetting the voting process to prevent tampering.
- **Result Declaration**: Outputs final vote counts for each candidate after voting concludes.
- **Reset Functionality**: Resets the entire system for subsequent voting sessions.

2.2 Modules and Their Functionalities

1. **Control Unit:**
    - Manages the overall operation flow, including initialization, voting, result display, and reset.
    - Implements state transitions based on user/admin inputs.
2. **Authentication Module:**
    - Verifies voter eligibility through predefined logic.
    - Prevents unauthorized access by validating input data.

3. **Vote Casting Module:**
    - Maps input buttons to specific candidates.
    - Handles vote registration and prevents multiple votes from a single input.
4. **Vote Counter Module:**
    - Stores vote counts securely in registers.
    - Ensures accurate vote tallying and supports real-time updates.
5. **Display Module:**
    - Visualizes the system state (e.g., idle, voting, or results mode).
    - Shows vote counts in binary format for candidates after the session.
6. **Admin Module:**
    - Provides controls to manage session transitions (start, end, reset).
    - Protects the integrity of the voting process.

2.3 Implementation Details

The EVM is designed using modular Verilog code, ensuring scalability and reliability. The primary modules are connected via signals, allowing communication between components.

1. **Button Control Module**
    - Debounces button presses to avoid unintentional multiple votes.
    - Ensures votes are logged only when the button is pressed for a specified duration.
    - Implements a 31-bit counter for timing the button press.
2. **Vote Logger Module**
    - Maintains a tally of votes in 8-bit registers for each candidate.
    - Increments vote counts only when in voting mode (Mode 0).
    - Uses hierarchical logic (if-else) to prioritize valid votes.
3. **Mode Control Module**
    - Operates in two modes:
        - **Mode 0:** Lights up LEDs to indicate a valid vote was cast.

  - **Mode 1:** Displays vote counts for each candidate in binary format using LEDs.
- Utilizes an internal counter to time LED states and prevent ambiguity.

4. **System Integration**
   - Combines the above modules into a single voting machine block.
   - Uses shared signals (e.g., valid votes, candidate-specific counters) for module interconnectivity.
   - Supports a scalable number of candidates by extending input/output configurations.

3 State Diagram

- T0(Start)
- T1(Authentication)
- T2(flag)
- T3 = f = 1
- T4(Mode)
- T5(Sufficient Time)
- T6(Result)
- T7(Vote Display)

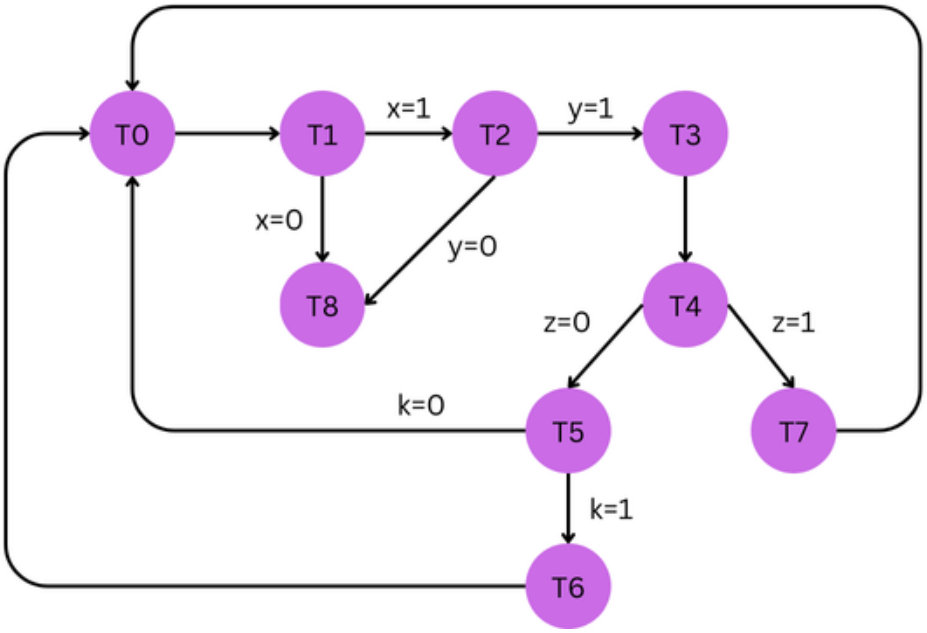

Figure 3.1: State Diagram

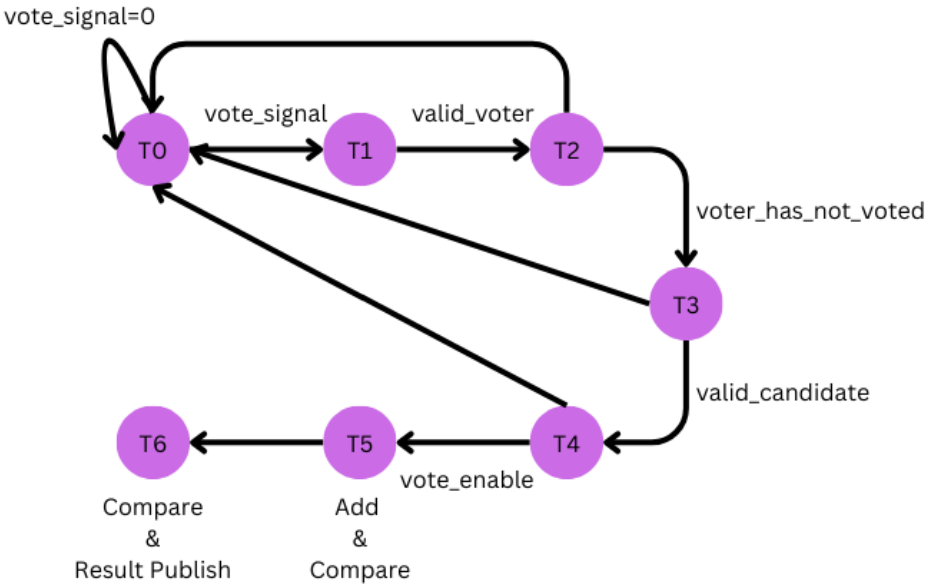

Figure 3.2: Updated State Diagram

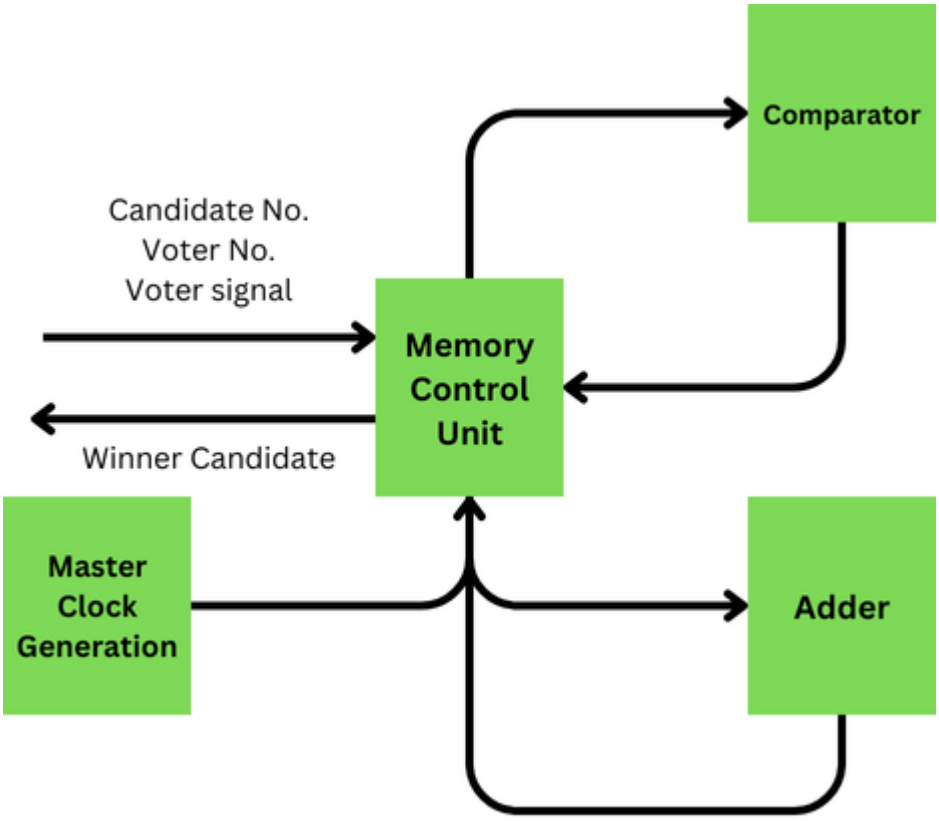

Figure 3.3: Memory Control Unit

- **T0 (Start):** The initial state where the EVM system is in standby or ready to begin.
  - **Transitions:**
    - Moves to **T1 (Authentication)** when a voter initiates the process.
- **T1 (Authentication):** This state handles the voter authentication process, ensuring only authorized voters proceed.
  - **Conditions:**
    - If $x = 1$ (authentication successful), the system transitions to **T2 (Flag)**.
    - If $x = 0$ (authentication failed), it loops back to **T0 (Start)**.
- **T2 (Flag):** This state checks or sets the flags necessary for voting.
  - **Conditions:**
    - If $y = 1$, it moves to **T3 (Voting)**.
    - If $y = 0$, it transitions to **T8 (Failure)**.
- **T3 (f=1):** This state indicates that voting is ready to proceed.
  - **Transitions:**

    - Goes to **T4 (Mode)** for mode selection.

- **T4 (Mode):** The mode-selection state, where either the admin or voter selects operations.
  - **Conditions:**
    - If $z = 0$, it transitions to **T5 (Sufficient Time)**.
    - If $z = 1$, it transitions to **T7 (Vote Display)**.

- **T5 (Sufficient Time):** Ensures there is adequate time for voting or any admin operation before concluding.
  - **Transitions:**
    - Moves to **T6 (Result)** to display the results.

- **T6 (Result):** Displays the final vote counts and locks the results, ending the session.

- **T7 (Vote Display):** Displays votes or provides a summary based on the selected mode.

- **T8 (Failure):** A fallback or error state when any critical condition fails (e.g., invalid flag or authentication).
  - **Transitions:**
    - Loops back to **T0 (Start)** for reinitialization.

4 Flow Diagram

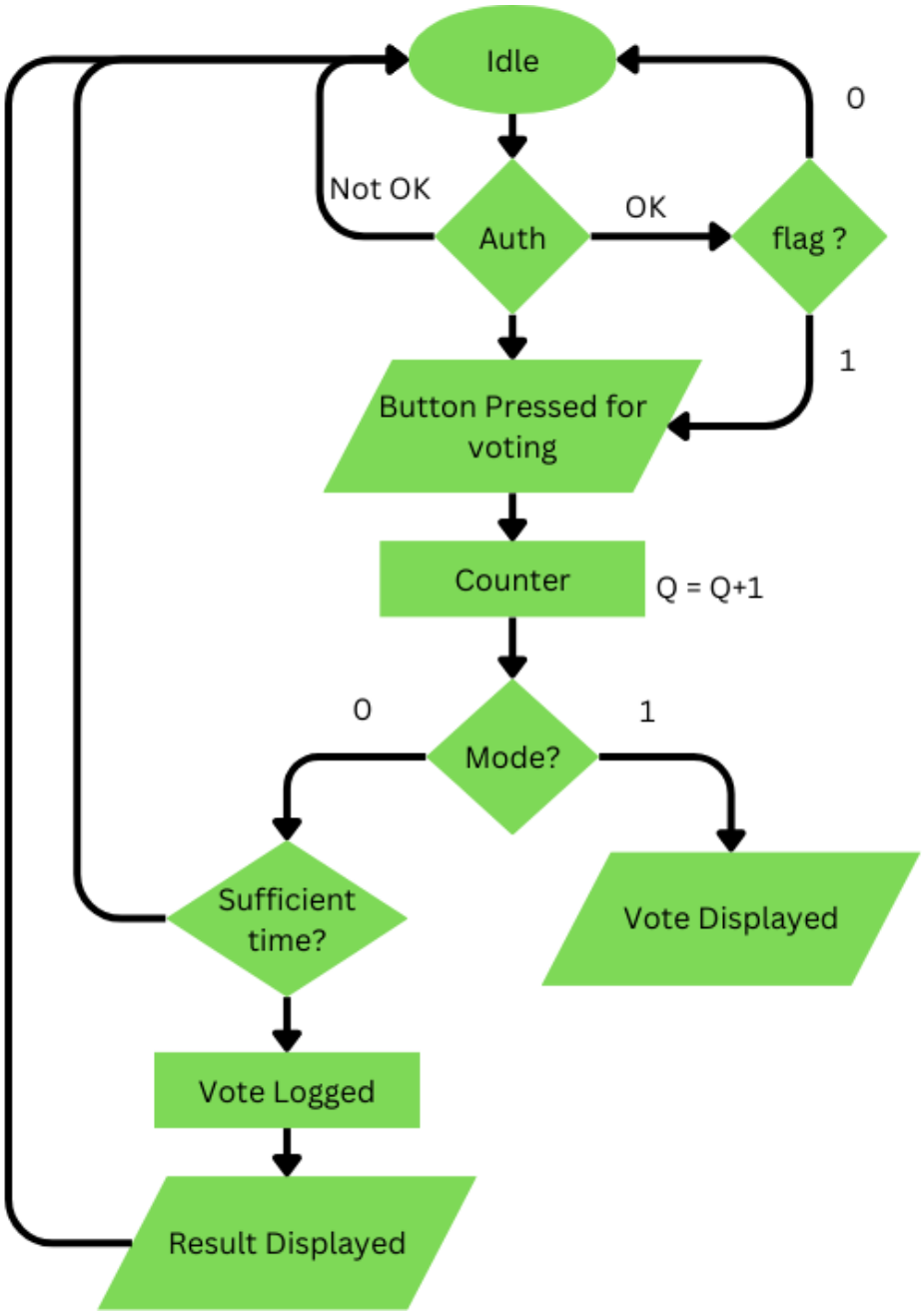

Figure 4.4: Flow Diagram

5 Conclusion

In this project, we have designed and implemented a simple voting machine system using Verilog, which includes several essential modules: Button Control, Vote Logger, Mode Control, and System Integration. Each module plays a crucial role in ensuring the correct operation of the voting system, including debouncing button presses, logging votes, and

controlling the display of results through LEDs.
The design ensures a smooth and accurate voting process by utilizing a 31-bit counter for timing, an 8-bit register for vote counting, and two operational modes: voting and displaying the vote counts. The system has been integrated in a way that supports scalability, allowing the addition of more candidates if necessary.
Through this project, we have demonstrated the practical application of Verilog in designing a digital voting system, providing a foundation for future enhancements and improvements, such as the integration of more advanced features like voter authentication, result encryption, and remote monitoring.

6 References

1. S. Smith, *Digital Design and Verilog HDL*, 3rd ed. New York: McGraw-Hill, 2018.
2. ”Verilog HDL for FPGA Design,” `https://www.examplewebsite.com/verilog-hdl`, accessed Dec. 2024.
3. ”Digital Logic and FPGA Design Techniques,” `https://www.anotherexample.com/digital-logic-fpga`, accessed Dec. 2024.